

IT UNIVERSITY OF COPENHAGEN

MSc in Computer Science
DevOps, Software Evolution and Software Maintenance
KSDSESM1KU

Group L

Email:	Name:
caft@itu.dk	Christina Aftzidis
dhel@itu.dk	Daniel Held-Hansen
fsan@itu.dk	Frederico Alves
lrpe@itu.dk	Laurids Robert Holme Pedersen
labp@itu.dk	Laurits Brok Pedersen
rakt@itu.dk	Rakul Maria Hjalmarsdóttir Tórgarð

May 29, 2025

Contents

1	System's Perspective	2
1.1	Design and Architecture of our ITU-MiniTwit System	2
1.2	Dependencies	2
1.3	Interactions of Subsystems	3
1.4	Current State of the System	4
2	Process' perspective	5
2.1	Our CI/CD pipeline	5
2.1.1	Continuous Integration	5
2.1.2	Continuous Delivery	7
2.2	Monitoring	8
2.3	Logging	9
2.4	Security	10
2.5	Scaling and Updates	11
2.5.1	Hosting and Infrastructure with DigitalOcean	11
2.5.2	Terraform for Infrastructure Management	11
2.5.3	Docker Swarm for Container Orchestration	11
2.5.4	Overall Infrastructure	11
2.6	Usage of AI-assistants	12
3	Reflection Perspective	12
3.1	Resolving Token Exchange Issues in Docker Swarm Setup	12
3.2	Challenges in Monitoring System Stability with Prometheus and Grafana	13
3.3	Lessons from an Easter Outage	13
3.4	DevOps culture	13
A	Appendix	16
A.1	Sonarcube report	16
A.2	Security Assesment - already enforced practices	17

1 System’s Perspective

1.1 Design and Architecture of our ITU-MiniTwit System

Our Minitwit application consists of two parts, a User Interface and an API, running in a Docker Swarm [9] on DigitalOcean [8] provisioned by Terraform [17]. Data is stored in a PostgreSQL [25] database.

We had a brainstorming session where we discussed what languages could be considered for refactoring. Constraints for the chosen language was it should be stable, well established in the industry, with extensive documentation and an active community.

The following languages were proposed: Julia, Rust, JavaScript, Nim, Ruby, Elixir and Go. Using Dot Voting [21], the choice was narrowed down to JavaScript and Ruby which both fulfilled our constraints. We chose Ruby because we wanted to learn a new language and we liked the simplicity of Ruby-Sinatra [3].

The professors initially recommended Vagrant [18] to provision our droplets. When implementing Docker Swarm, we had to replace the Vagrantfile with a bash script (see Section 2.5), so we knew that we could not revert to Vagrant for our Infrastructure as Code (IaC) solution. We refactored to Terraform because it ensures idempotency by design and is used in the industry.

1.2 Dependencies

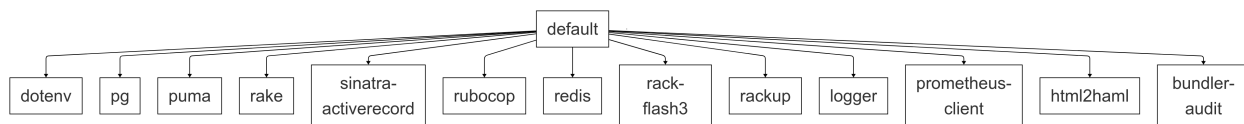


Figure 1: Graph of our Gemfile dependencies

Our system consists of three parts: a Ruby application, a PostgreSQL database and a Redis database [28]. The Ruby application can be further split into four parts: server, site, data and monitoring, see Figure 1 for the dependency tree. For serving our application, we rely on Puma [26] and Rackup [27], and, *dotenv* [7] dependencies since we can manage our environment using a *.env* file.

The application itself is written using Sinatra, chosen for its easy to read documentation and seemingly being the only used alternative to Ruby on Rails. Additionally, we depend on *html2haml* and *haml* for templating our HTML files and *rack-flash3* for pop-ups when accessing our application through the browser. Lastly, we use *bcrypt* [19] to encrypt passwords before storing them.

For data management we chose PostgreSQL since it was easier to set up than MySQL. Moving to a Docker Swarm based setup (see Section 2.5.3), we introduced a Redis cache to keep track of user sessions. Switching database providers was easy using Sequel [10], an object-relational mapping (ORM) library.

Finally for monitoring and logging we depend on the *prometheus-client* [1] and *logger* [30] gems for scraping and logging our application.

1.3 Interactions of Subsystems

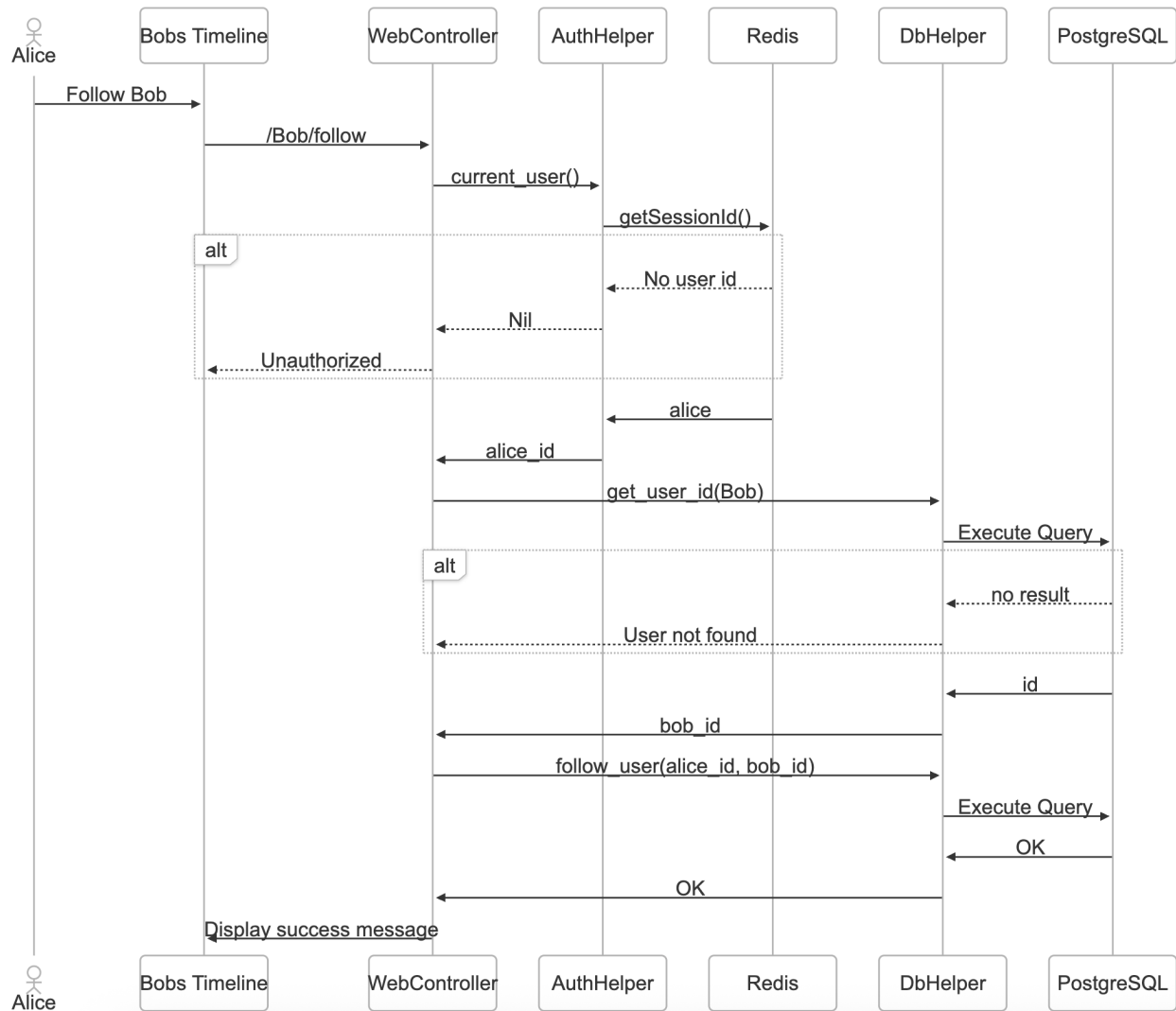


Figure 2: Sequence diagram of a follow user request traversing our system

The sequence diagram in Figure 2 illustrates the process of a user (Alice) initiating a "Follow Bob" action within a web application via the frontend. The WebController is responsible for handling user-initiated actions. For requests, it first ensures that the user is authenticated if required and then delegates to the DbHelper to retrieve or update information. An error message is displayed in the frontend if any errors occur.

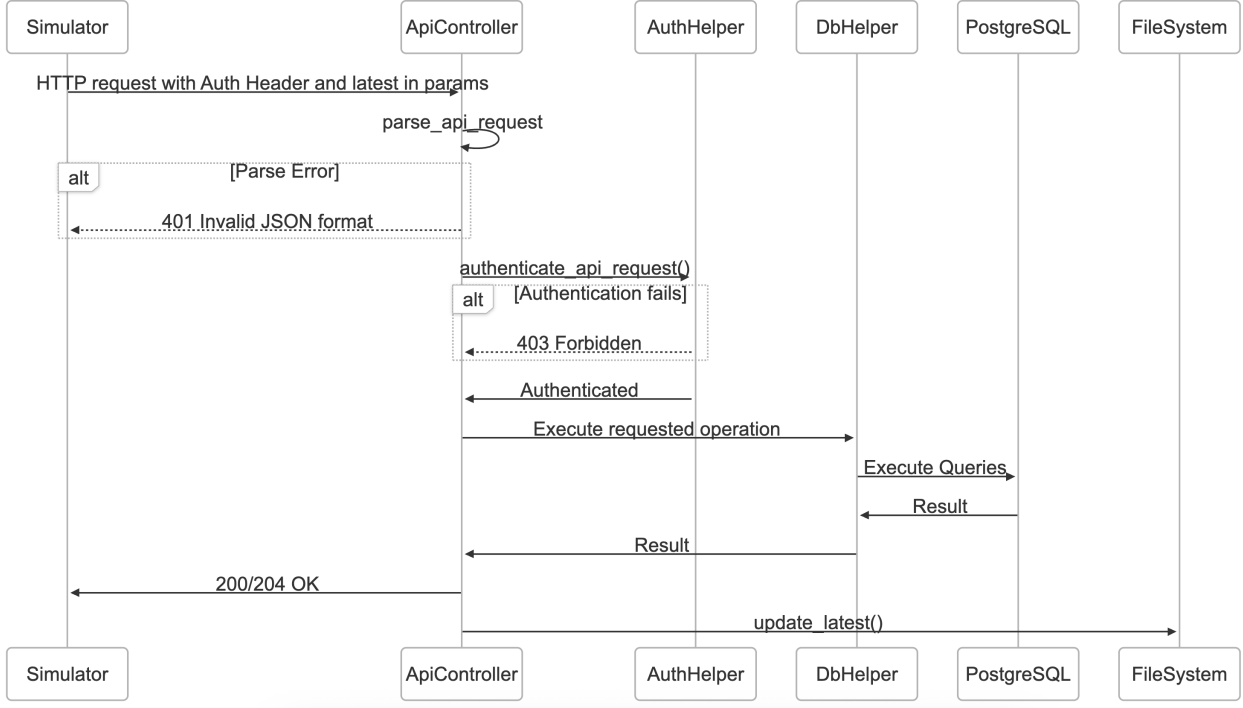


Figure 3: Sequence diagram of simulator request traversing our system

The sequence diagram in Figure 3 illustrates an API request originating from the Simulator and received by the ApiController. The ApiController begins by parsing the parameters and then verifies whether the API key provided in the header is valid. Upon successful authentication, the controller performs the requested operation through the DbHelper, which handles data reads and updates in PostgreSQL. Once the results are retrieved, the controller responds to the Simulator with an appropriate status code and includes the retrieved data in the response body. Finally, the system updates the "latest" state by invoking `update_latest()`.

1.4 Current State of the System

The following static analysis tools were used throughout the project, in order to improve readability, security and to follow best-practices:

1. **SonarQube** [31]: Software as a Service (SaaS) code analysis tool designed to analyse code for maintainability, reliability, and security issues.
2. **RuboCop** [2]: A static code analyzer and linter. Enforces many of the guidelines outlined in the community Ruby Style Guide [5].
3. **Bundler Audit** [4]: Checks for vulnerabilities in our Ruby dependencies.
4. **Hadolint** [6]: Dockerfile linter that helps build best-practice Docker images.

SonarQube reports one reliability issue, three security hotspots and one maintainability issue, all deemed low priority by the team (see Appendix A.1).

Some Rubocop rules, such as class length, were disabled due to disrupting developer workflows (see [24]). Customizing and enabling the rules would improve the state of the system.

2 Process' perspective

2.1 Our CI/CD pipeline

To support agility we implemented a fully automated CI/CD pipeline 4 using GitHub Actions. This choice was made because of its seamless integration with GitHub and its free tier of 2,000 Linux runner minutes per month which met our operational requirements.

2.1.1 Continuous Integration

Every time a Pull Request (PR) is opened, a series of automated Continuous Integration(CI) workflows are triggered to enforce code quality, security, and compliance standards before merging to the main code base, see figure 5.

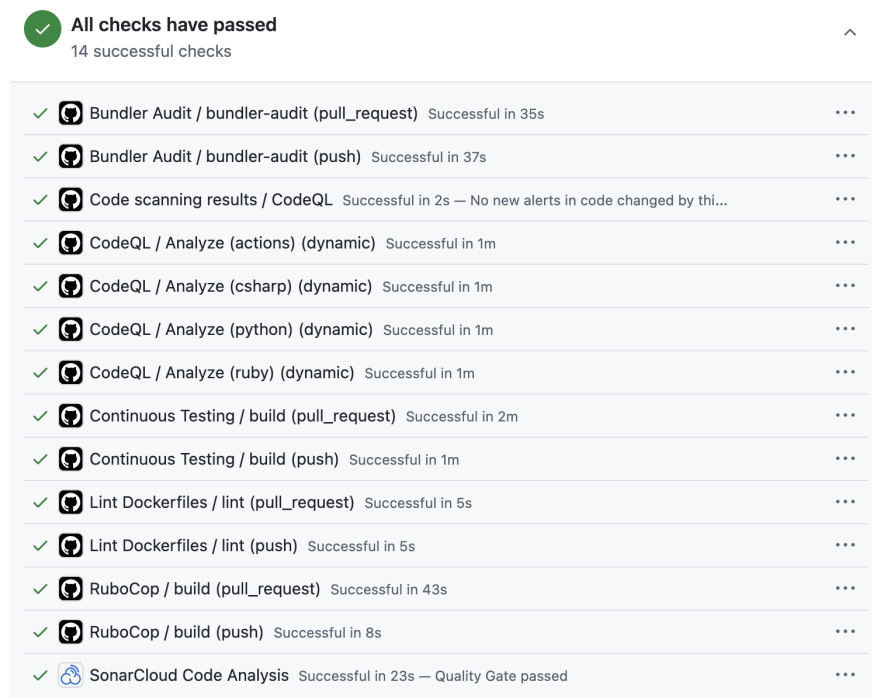


Figure 4: Quality Check on Continuous Integration on each Pull Request

All checks must pass before a PR can be merged, ensuring that only high-quality, secure code is integrated into the main branch.

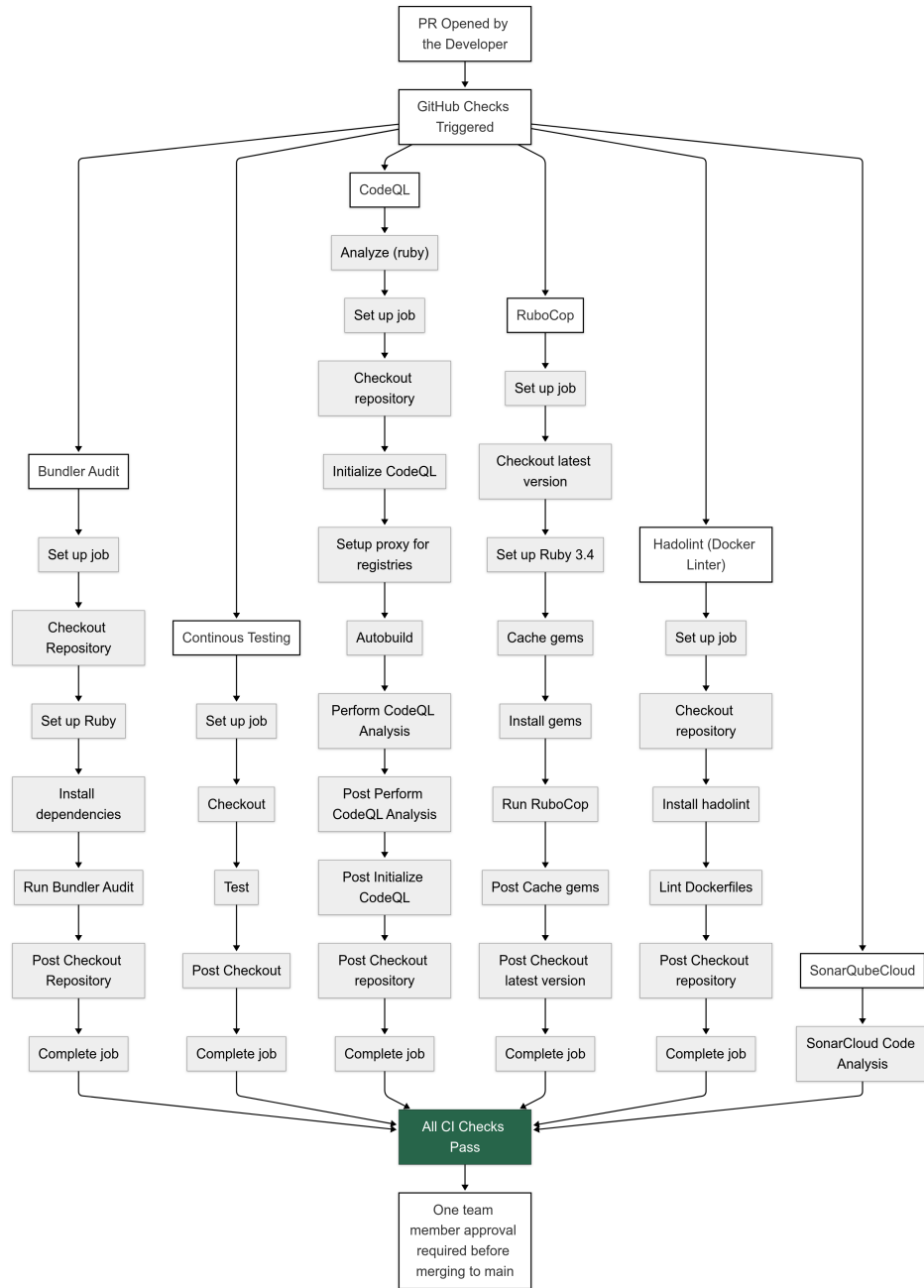


Figure 5: CI pipeline flow

2.1.2 Continuous Delivery

Once a PR is merged into the `main` branch, two workflows are triggered:

1. **Release Management** – A GitHub release is automatically created with a version tag, ensuring proper tracking and traceability of deployments.
2. **Continuous Deployment** – Builds and pushes Docker images (`minitwitimage`, `flagtoolimage`) to Docker Hub, sets up SSH access, and deploys the new version of the application to the target server environment in DigitalOcean.

After successful deployment, the application becomes immediately available in production.

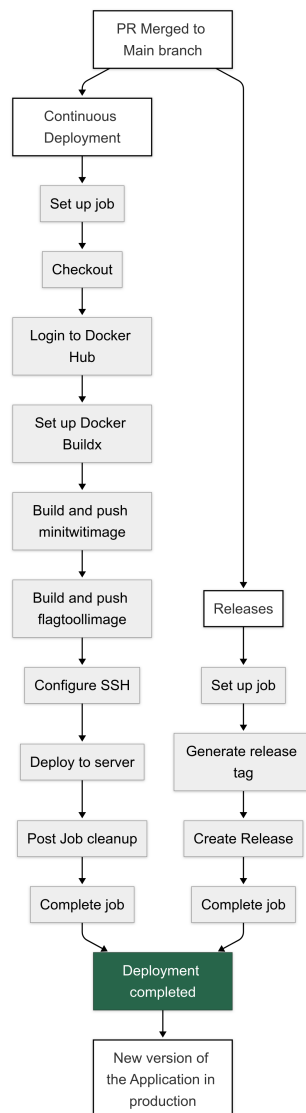


Figure 6: CD pipeline flow

2.2 Monitoring

The monitoring of our application is done with Prometheus and Grafana [14]. We chose these two systems due to them being industry standard, as well as the professors mentioning them as a possible option. Grafana and Prometheus are deployed in a Docker container situated on the manager node of our Docker swarm.

Prometheus is configured to target our Minitwit application. Initially, we were also monitoring Prometheus itself, but after multiple weeks of our Prometheus container continuously mysteriously crashing, we decided to reduce the scraping.

In addition to the default configuration for Grafana, we provide an admin user and password to our Dcompose file, preventing the resetting of credentials. Dashboards, datasources and alert rules are tracked in our Github repository and copied into the Grafana container upon creation. We discovered that this was needed shortly after implementing monitoring, when we realised that every deployment resets the dashboards.

Our dashboard (see Figure 7) keeps track of our server uptime and the average and maximum latency of requests, as well as requests per endpoint, status code, and ratio of return codes in percentage. Note that Figure 7 was not captured when the simulator was running, but instead after it had stopped using the simulator script provided. Server uptime was the first metric we monitored, as we found it crucial to know whether our application was up or not. In addition to having a visualisation in our dashboard, we also set up an alert to send a message to our Discord server whenever Minitwit is down. When the simulator started, our application got a few errors due to our return codes not matching the ones expected by the simulator. That and us accidentally resetting our database multiple times made us want to track the return codes of our application, as well as the requests per endpoint, so that we could spot issues faster. Shortly after implementing monitoring, our application started slowing down in comparison to our classmates'. This led to the addition of the two latency endpoints, allowing us to keep track of the response time of our system.



Figure 7: Grafana dashboard after running the test simulator

Our infrastructure is monitored using the built-in dashboard in DigitalOcean. It displays memory, CPU and disk usage.

2.3 Logging

The logging functionality in our system was implemented after the simulator ended using Ruby's native logging framework [29]. This built-in tool provides a simple yet flexible interface for outputting messages to standard output (STDOUT) or standard error (STDERR). Since our application runs within Docker containers, we leverage Docker's ability to capture STDOUT. This means all log messages are visible through the Docker logs, effectively mirroring what would appear in an interactive terminal session. This setup allows us to extract detailed runtime information from the application for monitoring and analysis.

To collect and visualize logs from our Docker containers, we integrated Loki [15] and Promtail [16] into our system. We chose these tools for log aggregation because they integrate seamlessly with our existing Prometheus and Grafana setup, allowing us to manage both metrics and logs within a unified Grafana interface. Promtail runs on every node and is responsible for forwarding Docker logs to Loki, as seen in Figure 8.

```
root@minitwit-swarm-leader:~# docker node ps
```

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE	ERROR	PORTS
noyv4trhxw5o	minitwit_database.1	postgres:17.4-bookworm	minitwit-swarm-leader	Running	Running 2 days ago		
9k5d1uy3dm7r	minitwit_grafana.1	grafana/grafana:11.5.2	minitwit-swarm-leader	Running	Running 2 days ago		
p5d5alqkafeh	minitwit_prometheus.1	prom/prometheus:latest	minitwit-swarm-leader	Running	Running 2 days ago		
rlygchj6jopr	minitwit_promtail.0lks8af1juzilc3emoipzpucl	grafana/promtail:latest	minitwit-swarm-leader	Running	Running 2 days ago		
vo4s287jvw4k	minitwit_redis.1	redis:7.2-alpine	minitwit-swarm-leader	Running	Running 2 days ago		

```
root@minitwit-swarm-leader:~#
```

Figure 8: Overview of running services in the Docker Swarm leader node. The image shows the active containers for the Minitwit application, including services for the database, Grafana, Prometheus, Promtail, and Redis.

Loki acts as the central log aggregation system. Unlike traditional logging systems that index the full log content, Loki is designed to index only the metadata with each log stream. This design choice, inspired by Prometheus, makes Loki more resource efficient and scalable. By configuring Loki as a data source in Grafana, we are able to query, filter, and visualize logs in real time.

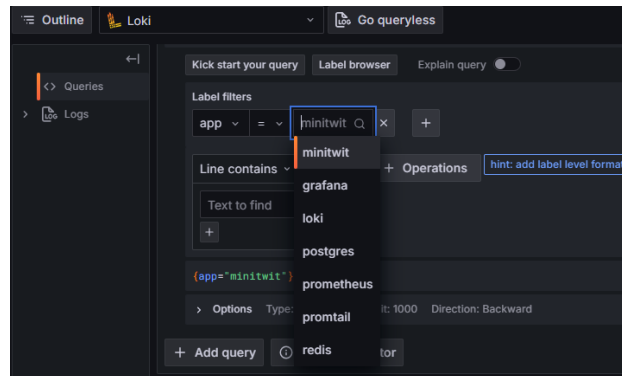


Figure 9: Grafana Explore interface showing log queries filtered by application label. This enables efficient selection and real-time inspection of logs from specific services.

To illustrate the full pipeline from application log generation to visualization, the Ruby code below shows how we emit log messages at different severity levels:

```
1 LOGGER = Logger.new($stdout)
2 LOGGER.info 'Home route hit'
3 LOGGER.warn 'Something might be off'
4 LOGGER.error 'Something went wrong'
```

These logs are captured by Docker, collected by Promtail and passed to Loki. As shown in Figure 10, they are then displayed in Grafana, where we can filter and view them in real time using the Explore interface.

```

> 2025-05-09 14:24:14.851 E, [2025-05-09T12:24:14.848879 #1] ERROR -- : Something went wrong
> 2025-05-09 14:24:14.851 W, [2025-05-09T12:24:14.848787 #1] WARN -- : Something might be off
> 2025-05-09 14:24:14.850 I, [2025-05-09T12:24:14.848618 #1] INFO -- : Home route hit

```

Figure 10: Logs generated using Ruby’s native logging framework[29].

2.4 Security

We performed a security assessment, identifying the following risk scenarios:

1. Attacker performs SQL injection on web application to download user data.
2. Attacker performs SQL injection on web application to delete user data.
3. An attacker gets access to our environmental secrets and uses them to get access to our VM, DB, dashboards or Discord.
4. An attacker exploits a vulnerability in a library.
5. An attacker leaks passwords leading to unrecoverable business reputation.
6. An attacker encrypts our DB leaving us unable to retrieve the DB.
7. An attacker guesses or brute-forces weak passwords, gaining unauthorised access to users’ accounts.

Each scenario was analysed, by discussing the impact and the probability of it. Finally, each scenario was mapped onto a risk matrix, see table 1. Using the risk matrix, it was concluded that scenario 4 and 7 was the most relevant to address. The impact of scenario 4 happening, really depended on what the attacker would exploit and what malicious things are done. As the scenario was certain to happen, we hardened the security of our system by implementing the following:

1. Enable Dependabot [12] to keep dependencies secure and up-to-date.
2. Enable CodeQL analysis / Code Scanning [11] to find security vulnerabilities and errors in our repository.
3. Enable Secret Protection and Push protection [13] which blocks commits that contain supported secrets.

A proposed solution for scenario 7, was to setup a timeout after an amount of faulty tries, by placing a user in a lockdown mode with limited capabilities. We also discussed setting up 2FA/MFA, however this was deemed out of scope. For security measures that were already enforced, see appendix A.2.

Impact ↑	Catastrophic	3	2, 6			4
	Critical		1, 5			4
	Marginal					4
	Negligible				7	4
	Insignificant					
		Rare	Unlikely	Possible	Likely	Certain
		Probability →				

Table 1: Risk matrix for security assesment.

2.5 Scaling and Updates

Our infrastructure leverages several technologies designed to ensure effective scaling and upgrade processes. Below, we explain our technology stack and strategies to optimize scaling capabilities and upgrade reliability.

2.5.1 Hosting and Infrastructure with DigitalOcean

We use DigitalOcean’s virtual machines (VMs) for our hosting infrastructure due to their free credits through GitHub Education and the recommendation by the professors. DigitalOcean allows us to quickly spin up servers/droplets or scale existing resources vertically or horizontally.

2.5.2 Terraform for Infrastructure Management

Terraform was chosen for our Infrastructure as Code (IaC) solution. Using Terraform, we declaratively manage our infrastructure, ensuring consistency across deployments. Terraform simplifies the scaling process, as infrastructure changes are defined in code and applied systematically. It also streamlines upgrades and configuration changes by clearly maintaining state. Our Terraform state is securely stored in DigitalOcean Spaces, which ensures centralized, reliable, and collaborative management of infrastructure changes.

2.5.3 Docker Swarm for Container Orchestration

We use Docker Swarm for managing our containerized applications, which provides straightforward container orchestration suitable for our project scale. Docker Swarm allows easy horizontal scaling of services by simply increasing replica counts. For instance, our application is replicated three times, enabling load balancing and fault tolerance.

Docker Swarm supports zero-downtime updates through rolling updates, updating one replica at a time with start-first ordering, ensuring new containers start successfully before older ones are shut down. This guarantees uninterrupted service availability during upgrades.

2.5.4 Overall Infrastructure

The overall infrastructure is illustrated in Figure 11. Terraform is responsible for creating and maintaining a distributed Docker Swarm architecture with three node types:

1. **Leader manager**

- (a) Maintains the cluster state and replicates updates to all other managers.

2. **Managers**

- (a) Orchestrate container deployments and ensure the desired state is upheld across all worker nodes.
 - (b) Host our logging, monitoring, and database services.
 - (c) Participate in the Raft consensus protocol [22] to keep the cluster state consistent in the event of leader failure.

3. **Workers**

- (a) Run instances of our replicated Minitwit service.

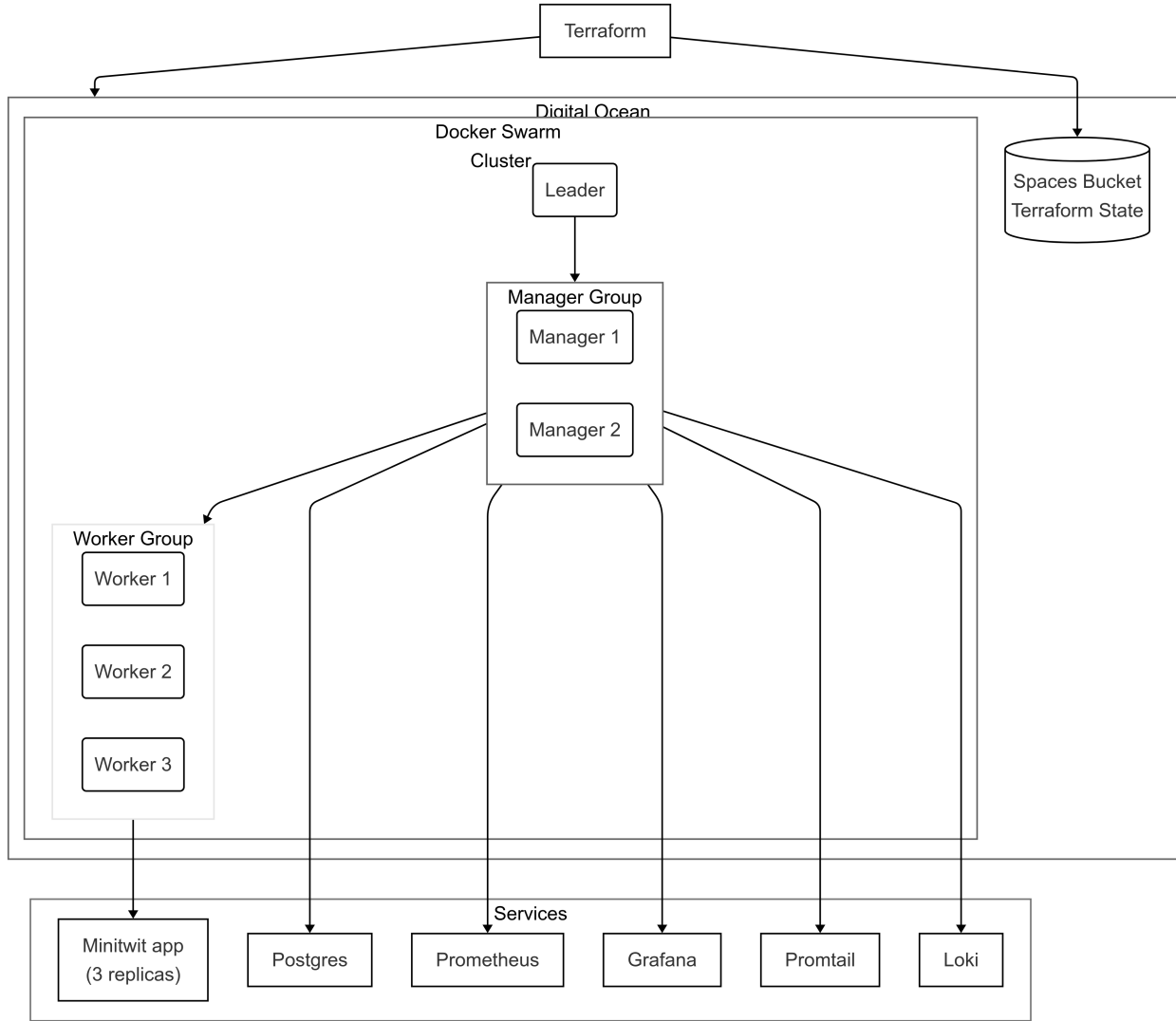


Figure 11: High-Level Infrastructure Diagram

2.6 Usage of AI-assistants

Throughout the project, we used generative AI, specifically ChatGPT, Claude and GitHub CoPilot, to assist us with programming tasks such as refactoring code. In the case of refactoring, using LLMs made the process faster but we still needed to do a lot of manual work since the solution proposed by the LLMs did not work out of the box. When setting up Docker Swarm, ChatGPT insisted it was easily done with a Vagrant file, but that turned out to be false and we realised hours later that a shell script was better suited.

3 Reflection Perspective

3.1 Resolving Token Exchange Issues in Docker Swarm Setup

We initially used a combination of Vagrant and shell scripts to provision and configure our VMs. The Vagrantfile handled the creation and provisioning of VMs, while the shell scripts managed environment configuration.

Integrating Docker Swarm into this workflow came with significant challenges. Our approach was to initiate a Docker Swarm within the shell script invoked by the Vagrantfile, however this introduced difficulties when

retrieving the token generated by `docker swarm init` to connect workers to the manager. After exploring several options, we ultimately moved away from the Vagrant-based setup. Instead, we used shell scripts to provision our infrastructure via the Digital Ocean API, which is better maintained and offers robust configuration options, for example it allows SSH keys to be passed as parameters during VM creation. Towards the end of the course, we further improved our deployment process by adopting Terraform. This provided a more efficient IaC solution.

3.2 Challenges in Monitoring System Stability with Prometheus and Grafana

As briefly mentioned in 2.2, monitoring our system was a near constant struggle throughout this project. Introducing Prometheus and Grafana to our stack led to our application crashing several times and our communication channel getting flooded with alerts from Grafana about Prometheus being down. Simultaneously, we were noticing an increased use of CPU resources. We did not think these two to be related initially and we were investigating other potential reasons for the increasing CPU usage, but when we decided to put off on restarting our Prometheus instance, we noticed that the CPU usage of our droplet stayed consistently low (see [23]).

We spent weeks looking into this issue, both by looking for solutions online and by consulting with other groups, but the solutions found did not work for us. Finally, we decided to (1) not expose the Prometheus port to the public to prevent unauthorised people from querying Prometheus data, (2) limit the CPU usage of the Prometheus container to a maximum of 20%, and (3) increase the scrape interval from 15 seconds to 60 seconds. These three changes seemed to solve the issue, though we did not realise that by doing (3) on the VM directly, that change would get overwritten at next deployment, leading us to puzzle over a Grafana dashboard that sometimes displayed no data due to queries timing out. To this day, we have not figured out what caused this issue in the first place, as there were groups with an identical setup that did not experience this. We might have identified the issue earlier if our logging setup had been operational sooner, which would have provided valuable insight.

3.3 Lessons from an Easter Outage

The day before the start of the Easter break, our production system transferred from being on a single droplet to a Docker Swarm setup. We came back and noticed that our application had been down for the entirety of the break, including the monitoring system responsible for notifying us. Simulator errors doubling was a stress-factor, which led to us understanding why so many companies schedule production freezes.

3.4 DevOps culture

Our DevOps culture was oriented on implementing the Three Ways principles from The DevOps Handbook [20, Chapter 1]. We implemented *left-to-right flow* using GitHub's issue tracker, kanban board and integration of a CI/CD pipeline. *Right-to-left* flow was achieved using monitoring and logging, assuring that we can swarm the problems as soon as they are detected. Having the responsibility of keeping a website continuously up is a full-time job and was quite stressful to manage alongside other courses.

We practiced the *Third Way* in our team through our ways of working. We made use of pair programming, did mental check-ins before we started working and created a culture of trust. Issues were self-assigned by team members, allowing people to work on what found interesting, resulting in a self-organised and motivated team.

References

- [1] Prometheus Authors. *prometheus-client RubyGem (Version 0.4.2)*. <https://rubygems.org/gems/prometheus-client/versions/0.4.2?locale=en>. Accessed: 2025-05-23. 2025. URL: <https://rubygems.org/gems/prometheus-client/versions/0.4.2?locale=en>.
- [2] Bozhidar Batsov and RuboCop contributors. *RuboCop*. en. 2025. URL: <https://rubocop.org/> (visited on 05/23/2025).
- [3] Blake Mizerany. *Sinatra*. Accessed: 2025-05-23. 2025. URL: <https://sinatrarb.com/>.
- [4] Hal Brogidan. *bundler-audit*. en. 2025. URL: <https://github.com/rubysec/bundler-audit> (visited on 05/23/2025).
- [5] Ruby Community. *Ruby Style Guide*. en. 2025. URL: <https://rubystyle.guide/> (visited on 05/23/2025).
- [6] Hadolint contributors. *Hadolint*. en. 2022. URL: <https://github.com/hadolint/hadolint> (visited on 05/23/2025).
- [7] Bobby DeVeaux and contributors. *dotenv RubyGem*. <https://github.com/bkeepers/dotenv>. Accessed: 2025-05-23. 2024.
- [8] DigitalOcean. *DigitalOcean Documentation*. Accessed: 2025-05-23. 2025. URL: <https://docs.digitalocean.com/>.
- [9] Docker Inc. *Swarm mode*. Accessed: 2025-05-23. 2025. URL: <https://docs.docker.com/engine/swarm/>.
- [10] Jeremy Evans. *Sequel: The Database Toolkit for Ruby*. <https://sequel.jeremyevans.net>. Accessed: 2025-05-23. 2024.
- [11] GitHub. *About code scanning - Introduction to CodeQL*. Accessed: 2025-05-23. 2024. URL: <https://docs.github.com/en/code-security/code-scanning/introduction-to-code-scanning/about-code-scanning>.
- [12] GitHub. *About supply chain security - What is Dependabot?* Accessed: 2025-05-23. 2024. URL: <https://docs.github.com/en/code-security/supply-chain-security/understanding-your-software-supply-chain/about-supply-chain-security#what-is-dependabot>.
- [13] GitHub. *Secret scanning patterns - Supported secrets*. Accessed: 2025-05-23. 2024. URL: <https://docs.github.com/code-security/secret-scanning/secret-scanning-patterns#supported-secrets>.
- [14] Grafana Labs. *Grafana: The Open and Composable Observability Platform*. <https://grafana.com/>. Accessed: 2025-05-23. 2025.
- [15] Grafana Labs. *Loki Documentation*. Accessed: 2025-05-23. 2025. URL: <https://grafana.com/docs/loki/latest/>.
- [16] Grafana Labs. *Promtail Agent Documentation*. Accessed: 2025-05-23. 2025. URL: <https://grafana.com/docs/loki/latest/send-data/promtail/>.
- [17] HashiCorp. *Terraform by HashiCorp*. Accessed: 2025-05-23. 2025. URL: <https://developer.hashicorp.com/terraform>.
- [18] HashiCorp. *Vagrant by HashiCorp*. Accessed: 2025-05-23. 2025. URL: <https://developer.hashicorp.com/vagrant>.
- [19] Kelektiv. *node.bcrypt.js: bcrypt for Node.js*. <https://github.com/kelektiv/node.bcrypt.js>. Version 6.0.0, Accessed: 2025-05-23. 2025.
- [20] Gene Kim et al. *The DevOps handbook: How to create world-class agility, reliability, & security in technology organizations*. It Revolution, 2021.
- [21] Lucid Software. *How dot voting helps with group decision-making*. Accessed: 2025-05-23. 2024. URL: <https://lucid.co/blog/dot-voting>.
- [22] Diego Ongaro and John Ousterhout. “In search of an understandable consensus algorithm”. In: *2014 USENIX annual technical conference (USENIX ATC 14)*. 2014, pp. 305–319.

- [23] Only-Smiles. *Minitwit high CPU usage · Issue #147 · Only-Smiles/DevOps-2025*. en. 2025. URL: <https://github.com/Only-Smiles/DevOps-2025/issues/147> (visited on 05/15/2025).
- [24] Only-Smiles. *Rubocop Configuration*. en. 2025. URL: <https://github.com/Only-Smiles/DevOps-2025/blob/main/.config/.rubocop.yml> (visited on 05/23/2025).
- [25] PostgreSQL Global Development Group. *PostgreSQL: The World's Most Advanced Open Source Relational Database*. Accessed: 2025-05-23. 2025. URL: <https://www.postgresql.org/>.
- [26] Puma Contributors. *Puma: A Ruby/Rack Web Server Built for Parallelism*. Accessed: 2025-05-23. 2025. URL: <https://github.com/puma/puma>.
- [27] Rack Contributors. *rack/rackup: Command-Line Interface for Rack Applications*. Accessed: 2025-05-23. 2025. URL: <https://github.com/rack/rackup>.
- [28] Redis Ltd. *Redis: The Real-time Data Platform*. Accessed: 2025-05-23. 2025. URL: <https://redis.io/>.
- [29] Ruby Core Team. *Ruby Logger Library*. <https://github.com/ruby/logger>. Accessed: 2025-05-22. 2025.
- [30] Naotoshi Seo and SHIBATA Hiroshi. *Logger Ruby Gem v1.6.0*. <https://rubygems.org/gems/logger/versions/1.6.0>. Version 1.6.0. A simple logging utility for outputting messages in Ruby. Released November 7, 2023. Nov. 2023.
- [31] Sonar. *Sonarqube Cloud*. en. 2025. URL: <https://github.com/apps/sonarqubecloud> (visited on 05/23/2025).

A Appendix

A.1 Sonarcube report

Only Smiles > DevOps-2025 > 1 dev

SummaryIssuesSecurity HotspotsMeasuresCodeActivity

Main Branch Summary

319 Lines of Code • Last analysis 6 days ago • aa94c75a

Quality Gate 

 **Passed**

New Code

Overall Code

Security

0 Open Issues



Reliability

1 Open Issues



Maintainability

1 Open Issues



Accepted Issues

0



Coverage

A few extra steps are needed for SonarQube Cloud to analyze your code coverage.
[Set up coverage analysis](#)



Duplications

0.0%

No conditions set on 372 Lines



Security Hotspots

3

<!DOCTYPE html>

<html>

Add "lang" and/or "xml:lang" attributes to this "<html>" element

FROM mcr.microsoft.com/dotnet/core/runtime:3.1 as base

Replace "as" with upper case format "AS".

Review priority:  Medium

 Permission

2



This image might run with "root" as the default user. Make sure it is safe here.

The "ruby" image runs with "root" as the default user. Make sure it is safe here.

Review priority:  Low

 Insecure Configuration

1



Make sure this debug feature is deactivated before delivering the code in production.

3 of 3 shown

16

A.2 Security Assesment - already enforced practices

The VMs hosted on DigitalOcean droplets are only accessible via SSH. Public SSH keys are added through the DigitalOcean UI, ensuring that only authorized developers with the corresponding private keys can connect. Furthermore, we made use of DigitalOcean Space and GitHub Secrets to securely store our environment variables and sensitive configuration values, such as API keys and credentials, separating them from our codebase. This minimizes the risk of accidental exposure.

To secure communication between users and our servers, we configured DNS for our domain and enabled HTTPS using TLS certificates. This ensures that all transmitted data is encrypted and protected against interception and Man-in-the-Middle attacks.

For database security, we moved our database to a private internal network, making it inaccessible from the public internet.

Finally, we enhanced credential security by using autogenerated strong passwords for all service accounts and system users. This reduces the likelihood of successful brute-force attacks and improves overall system resilience.